



Carrera:

**Licenciatura en Programación
y web máster**

Docente:

Mario Alonso Segovia Gutiérrez

Materia:

Programación Avanzada

Alumno(s):

Leylani Yael Marín Jiménez

Fecha:

19/07/2026

Cuadro comparativo: Sesiones, Cookies y JSON Web Tokens (JWT)

Criterio	Sesiones (Sessions)	Cookies	JSON Web Tokens (JWT)
Definición	Mecanismo del lado del servidor que guarda el estado del usuario; el servidor genera un identificador único (session ID) que se envía al cliente, normalmente por medio de una cookie.	Pequeño archivo de texto que el servidor envía al navegador y que este almacena y reenvía en cada petición al mismo dominio.	Estándar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir información de forma segura entre partes, como un objeto JSON firmado digitalmente.
¿Dónde se almacena la información?	Los datos se guardan en el servidor (memoria, archivos o base de datos); en el cliente solo se conserva el identificador de sesión.	En el cliente (navegador del usuario).	En el cliente, comúnmente en localStorage, sessionStorage o una cookie.
¿Quién administra la información (cliente o servidor)?	El servidor administra y controla toda la información; el cliente solo conserva el ID de sesión.	El cliente almacena la cookie, pero el servidor decide su contenido, duración y atributos al crearla.	El cliente almacena y envía el token; el servidor solo lo firma y verifica, sin necesidad de guardar estado por usuario.
Nivel de seguridad	Alto: los datos sensibles nunca viajan al cliente, solo el identificador de sesión.	Medio-bajo si no se configura correctamente; depende de atributos como HttpOnly, Secure y SameSite.	Depende de la implementación: la firma garantiza integridad, pero el payload no está cifrado por defecto (solo codificado en Base64).
Persistencia de la información	Temporal; expira al cerrar el navegador o tras un tiempo de inactividad definido en el servidor.	Puede ser de sesión (se borra al cerrar el navegador) o persistente (con fecha de expiración definida).	Definida por el campo "exp" dentro del propio token; puede combinarse con refresh tokens para renovar el acceso.
Ventajas	Mayor seguridad al mantener los datos en el servidor; fácil de invalidar (cerrar sesión); permite almacenar mucha información.	Simplicidad de implementación; no requiere base de datos; útil para preferencias del usuario además de autenticación.	No requiere almacenamiento en el servidor (stateless); facilita el escalamiento horizontal; ideal para APIs, microservicios y autenticación entre dominios.
Desventajas	Consume recursos del servidor; dificulta el escalamiento horizontal sin almacenamiento compartido (Redis, base de datos); depende de que el cliente acepte cookies.	Tamaño limitado (~4 KB); se envía en cada petición, aumentando el tráfico; vulnerable si no se configuran bien sus atributos.	Difícil de invalidar antes de su expiración; si se roba es utilizable hasta que expire; el payload es legible si no se cifra.
Casos de uso	Aplicaciones web tradicionales con backend centralizado, paneles administrativos, sistemas de comercio electrónico.	Recordar preferencias del usuario (idioma, tema), carritos de compra, identificadores de sesión, analítica y rastreo.	APIs REST, aplicaciones SPA (React, Angular, Vue), microservicios, autenticación entre distintos servicios (SSO).
Ejemplo de aplicaciones donde se utilizan	Sistemas PHP con session_start(), paneles de administración de WordPress, aplicaciones Java con HttpSession.	Banners de consentimiento de cookies, carritos de tiendas en línea, Google Analytics.	Auth0, Firebase Authentication, inicio de sesión con Google/Facebook, APIs construidas con Node.js o Spring Boot.
Riesgos de seguridad más comunes	Secuestro de sesión (session hijacking), fijación de sesión (session fixation), robo del ID mediante XSS.	Cross-Site Scripting (XSS) para robo de cookies, Cross-Site Request Forgery (CSRF), interceptación en redes no cifradas.	Robo del token mediante XSS si se guarda en localStorage, uso del algoritmo "none", tokens con expiración demasiado larga.

Criterio	Sesiones (Sessions)	Cookies	JSON Web Tokens (JWT)
Medidas recomendadas para proteger la información	Usar cookies con atributos HttpOnly y Secure, regenerar el ID de sesión al iniciar sesión, definir tiempos de expiración cortos, usar HTTPS.	Configurar HttpOnly (evita acceso por JavaScript), Secure (solo HTTPS) y SameSite (evita CSRF); definir una expiración adecuada.	Usar algoritmos de firma seguros (por ejemplo RS256), definir tiempos de expiración cortos, preferir cookies HttpOnly sobre localStorage, implementar refresh tokens con rotación.